

师问智联助手-TQBot

小组成员：张思雨、唐欣妍、高佳宁、王书瑶

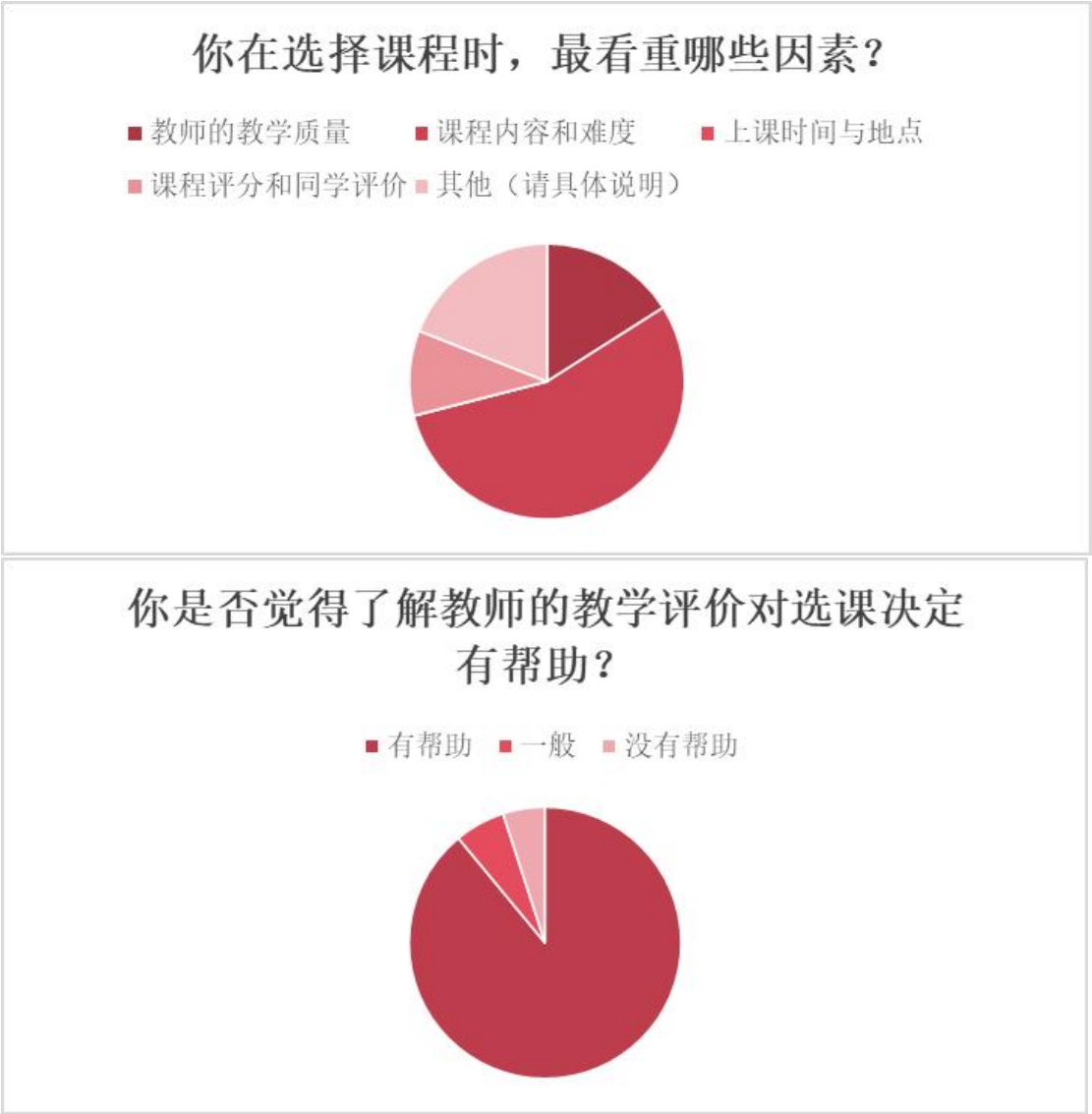
1 项目背景	错误！未定义书签。
2 设计思路	6
3 技术实现	7
3.1 前端页面	7
3.2 后端及 LANGCHAIN 实现	7
3.3 前后端交互	9
3.4 云端部署	10
4 团队分工	8

1 项目背景

在当代高等教育中，课程选择是学生学业规划中的关键环节，直接影响其学习兴趣与学术发展。然而，随着高校课程数量的不断增加以及教学资源的多样化，学生在选课过程中面临着诸多挑战。

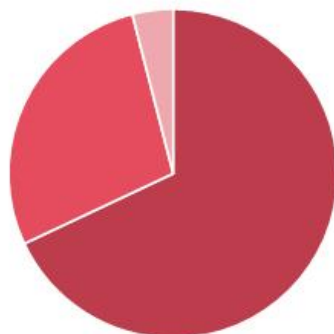
为更好地了解学生在选课过程中的主要需求和困境，我们开展了一项问卷调查，覆盖了课程选择中重要的影响因素、信息获取方式、对智能选课系统的期待等关键问题。调查结果验证了当前选课问题的普遍性，并为本项目的设计提供了直接的数据支持。

以下是问卷结果的部分展示：



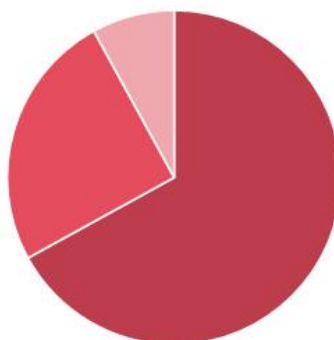
在选课过程中，是否存在你对课程信息获取困难的情况？

■ 经常 ■ 偶尔 ■ 从未



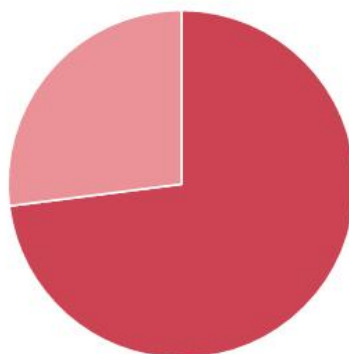
如果AI选课系统能够通过问答形式直接向你提供课程的教学评价，你认为该系统能否帮助你更快、更准确地选择合适的课程？

■ 能 ■ 一般 ■ 不能



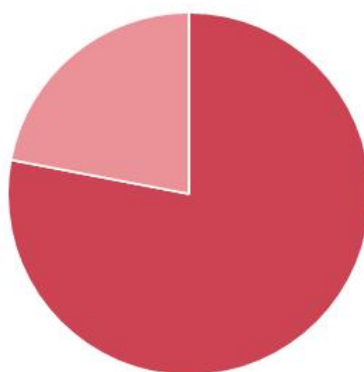
你是否希望有一个AI智能选课系统去帮助你选择
合适的课程？

■ 是 ■ 否



如果系统能保证评价数据的隐私保护，你是否愿
意提供你对教师的评价？

■ 是 ■ 否



问卷数据显示，55%的学生最关注课程内容与难度，16%的学生重视教学质量，这表明学生对于选课中课程适配度的需求十分显著。89%的学生认为了解教师的教学评价对选课有帮助，表明评价数据在决策中有重要作用。然而，尽管教学评价数据具有参考价值，学生普遍反映获取信息的过程存在障碍。问卷显示，68%的学生经常遇到课程信息获取困难，而仅有4%的学生从未遇到过困难，这一结果凸显了现有信息传递方式的局限性。数据的分散性和呈现形式的低效，可能是导致问题的主要原因。同时，调查还显示，73%的学生期待智能选课系统的帮助，而67%的学生认为通过问答形式的智能系统可以快速、准确地辅助决策，反映出智能选课系统具有较大的需求潜力。最后，78%的学生愿意在隐私保护的前提下提供对教师的评价，说明该系统的基础数据收集的可行性很强。

综上所述，目前高校在选课信息服务上的不足，尤其是在教学评估数据的高效利用方面，

迫切需要新的技术解决方案。与此同时，人工智能技术的发展为这一问题的解决提供了可能性。通过引入 AI 技术，建立一个以数据为驱动、以智能问答为核心的选课辅助系统，可以为学生提供更科学的决策依据，极大提升选课效率与决策质量。

基于此，本项目以“学生选课辅助”为目标，结合人工智能技术与高校教学评估数据，设计并实现了一个智能问答选课系统。该系统能够将学生的自然语言提问转化为 SQL 查询，提取数据库中的课程评价数据，并以自然语言的形式输出分析结果，为学生提供高效、直观的课程推荐与决策支持。

2 设计思路

首先，由于我们希望能建立一个智能系统，参考了常见的系统设计方法，同时从用户使用的便捷性和开发实现的高效性综合考虑，我们选择了以网页的形式呈现，这样便于我们调用 AI 接口、部署到云服务器，用户也可以通过多端访问。

进一步考虑网页的结构设计，我们采用了传统的网页实现方法，主要使用 html 语言编写前端代码用于呈现有优异视觉效果的面，在上面展示类似于 GPT 页面的 AI 问答助手，与用户建立对话——用户可通过页面上的对话框输入与课程或教师相关的问题。同时为了给用户们使用带来便捷性，我们还在页面侧边栏设计了推荐问题的展示，希望能给用户带来一些参考。

对于后端部分，我们首先选用 Flask 框架将其与前端连接起来，旨在以少量高效的代码实现前后端交互。同时，为了让 AI 能够返回针对性的回答，我们选用常用的 MySQL 数据库创建我们系统的知识库，在里面创建数据表存储教师授课内容、课程给分制度等能辅助学生了解该课程的信息。此外，我们希望用 langchain 接上 AI 接口，由此综合比较了多种大语言模型，发现 deepseek 的回答逻辑清晰、易于用户理解，同时其 API 的获取也十分方便，是实现我们智能回答助手的不二选择，因此接入了 deepseek 的 API 进行调用。

最后，由于希望能将我们的作品进行广泛地展示与被使用，我们将实现该系统的本地代码与数据库部署在云服务器上——此处选择了阿里云服务器，其价格实惠、可以供用户自行选择配置参数以针对性地部署不同项目，这些优势是我们选择的原因。

3 技术实现

3.1 前端页面

(1) 页面结构 (HTML)

- 使用 HTML 定义页面布局，分为顶部导航栏、右侧固定的侧边栏、主聊天区域和输入框区域。
- 聊天区域用于显示对话记录，输入框用于用户输入消息。

(2) 样式设计 (CSS)

- 页面通过内联样式实现基本的布局 and 美化。
- 布局采用固定侧边栏和滚动的主聊天框。
- 使用配色和悬停效果来增强交互体验（如按钮悬停变色）。

(3) 动态功能 (JavaScript)

- 使用 JavaScript 实现用户输入与 AI 回复的功能
 - 用户可以通过两种方式与系统交互：在输入框中直接键入问题，点击发送按钮提交问题。从推荐问题列表中选择一个问题，问题会被自动填充到输入框，用户只需点击发送即可。
 - 通过 fetch 向后端发送请求，接收 AI 回复并动态显示在页面。
- 在页面加载完成时，前端通过 window.onload 添加初始欢迎语（“你好，我是 TQBot，有什么可以帮到您的吗？”），为用户提供友好的初始互动。
- 区分用户消息和 AI 消息，通过样式调整对齐方式。

3.2 后端及 langchain 实现

(1) langchain 实现

为了接入大语言模型（在“2 设计思路”中我们指出，本项目使用的是 deepseek），需要在 python 中借助 langchain 库调用 deepseek 的 API。

- 登录 deepseek 官网，获得 API 密钥，以便后续获取相关服务：



- 配置大语言模型 llm, 设置回答内容多样性, 指定回答的模型, 明确接口:

```
#大语言模型, 下面都要用这个llm来回答
llm = ChatOpenAI(
    temperature=0.95,      #多样性
    model="deepseek-chat",
    openai_api_key=deepseek_key,
    openai_api_base="https://api.deepseek.com"
)
```

- 调用 langchain 库的内存缓存函数, 存储对话历史:

```
# 内存缓存
memory = ConversationBufferMemory(return_messages=True)
```

- 调用 langchain 库里的 ChatPromptTemplate 和 RunnablePassthrough.assign, 实现 get_sql_chain()函数, 生成一个任务链, 用于将用户问题转化为 SQL 查询。
- 再次利用 langchain 库里的 ChatPromptTemplate 和 RunnablePassthrough.assign, 生成完整任务链 (get_sql_result_chain()函数), 整合了 SQL 查询生成、数据库查询和答案生成, 生成最终回答。

(2) 数据库实现

为用户从多方面了解一位老师和课程, 需要用数据库建立相应的知识库。

- 在 MySQL 中创建数据表 education, 在里面存入教师名字、课程名称、通过率、均分等相关数据, 部分字段和行如下所示:

	record_id	teacher_id	teacher_name	course_id	course_name	semester_name	homework_load	pass_rate	average_score	difficulty
▶	1	101	张伟	1001	高等数学	2024春季	3.5	0.85	78.5	4.0
	2	102	李娜	1002	线性代数	2024春季	3.0	0.90	82.3	3.8
	3	103	赵明昊	1003	云计算系统	2024秋季	4.0	0.75	75.0	4.0
	4	104	赵敏	1004	物理学导论	2024秋季	2.5	0.88	85.6	4.2
	5	105	孙丽	1005	英语写作	2024春季	3.8	0.92	89.4	2.9
	6	106	周杰	1006	数据结构	2024秋季	4.2	0.80	76.5	4.1
	7	107	吴婷	1007	有机化学	2024春季	3.6	0.83	81.0	3.9
	8	108	郑凯	1008	宏观经济学	2024秋季	2.8	0.87	84.2	3.7
	9	109	冯雪	1009	心理学基础	2024春季	3.2	0.89	86.7	3.4

- 在本项目文件夹下编写 db_setting.py 函数，实现 python 与数据库的交互，设置用户名、密码、主机地址、端口号等，使代码运行时能自动连上该数据库，并编写 run_query()函数，实现对数据表的内容查询。

3.3 前后端交互

(1) 前端请求

- sendMessage() 方法首先收集用户输入，调用 addMessage('user', userMessage) 将用户的提问实时显示在聊天窗口中，提供即时的反馈体验。
- 前端通过 fetch 方法向后端指定的 API (http://127.0.0.1:5000/get_answer) 发送一个异步 HTTP POST 请求。
- 请求头中指定 Content-Type 为 application/json，请求体中封装用户的提问内容 ({ question: userMessage })，以 JSON 格式传输，确保数据结构清晰且可扩展。

(2) 后端处理

- 后端的 Flask 应用接收到来自前端的请求，通过 /get_answer 路由捕获 POST 请求。
- 在视图函数中，使用 request.json.get('question') 提取用户问题。
- 提取后的问题被传递给业务逻辑处理函数 get_sql_result_chain()，该函数负责查询数据库或调用相关模块来生成对用户问题的答案。
- 后端通过 jsonify 将生成的答案打包为标准的 JSON 格式，返回给前端。

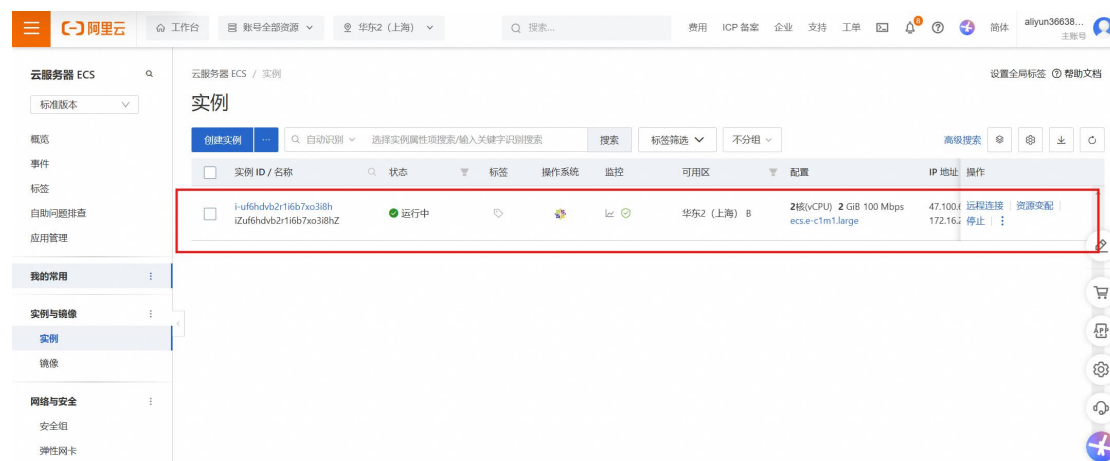
(3) 响应传递

- 前端接收到后端返回的 JSON 数据后，解析其中的 answer 字段。
- 调用 addMessage('ai', aiResponse) 将 AI 的回答内容动态更新到聊天窗口中。
- 整个交互流程的响应时间较短，用户感受到的是流畅的对话体验。

3.4 云端部署

(1) 服务器租用

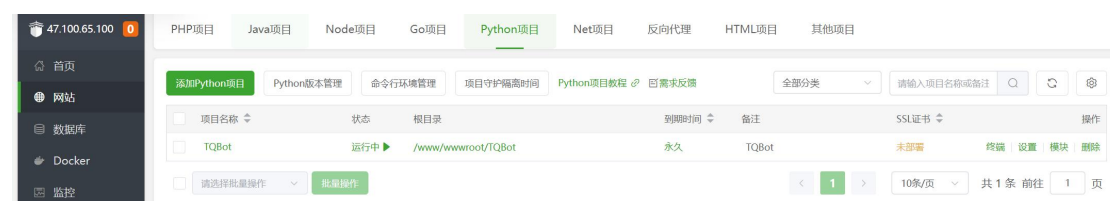
我们在阿里云官网租用了一台云服务器（基于 Centos7.9），以此获得供网页部署的云资源和访问网页的 ip 地址（47.100.65.100）：



(2) 网页部署

获得云服务器地址后，我们需要远程连接该服务器并将项目文件部署上去。此处我们使用了一个便于快速部署网页的工具——宝塔面板，在里面将本地的项目代码文件和数据库上传，并分别进行部署，如下所示：

● python 项目：



● MySQL 数据库：



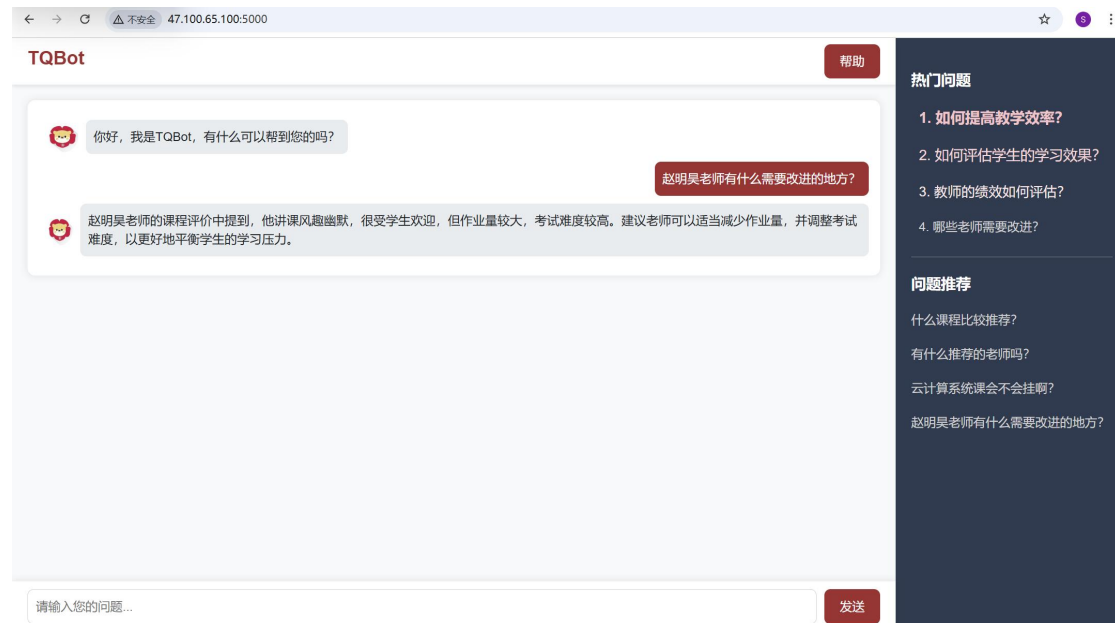
在部署时，我们将代码中出现本地地址的地方改为阿里云服务器地址，便于后端连上云服务器而非本地服务器。

同时，我们将部署时的域名设置为阿里云服务器地址 47.100.65.100，那么这样就可以

通过浏览器访问该地址+相应端口号（此处为 5000）便能看到我们的 TQBot 网页了。

- 在浏览器中通过云服务器地址访问：

我们的部署成功完成，如下所示：



——现在，您可在浏览器输入以下网址与我们的师问智联助手 TQBot 进行互动！

<http://47.100.65.100:5000/>

4 团队分工

我们小组成员分工明确，齐心协力，共同完成了这份作品~以下是我们的分工明细：

- 张思雨

- 设计和维护 MySQL 数据库结构

- 搭建和维护 Flask 后端服务

实现 API 接口，处理前端请求和数据交互

- 唐欣妍

- 集成 LangChain 和 DeepSeek 语言模型，实现自然语言处理功能

- 负责应用的部署和环境配置，将其部署在云服务器上，确保系统稳定运行。

- 高佳宁

- 功能与问题设计
 - 设计和实现用户界面，包括页面布局和交互设计
- 进行前端功能测试，确保界面和交互的正常运行

- 王书瑶

- 集成 LangChain 和 DeepSeek 语言模型，实现自然语言处理功能
- 处理模型调用和结果解析
- 验证 AI 模型集成的准确性和稳定性